Testing the cave search

The search makes three kinds of claim, and each has its own test. They are not interchangeable: passing one says nothing about the others.

claim	example	test	section
exact: this number is what the game produces	"this cavern is 55,830,955 blocks"	the game's own code	1
equivalent: this faster code gives the same numbers	"the new look kernel is the old one, three times faster"	old against new on the same inputs	2
better: this change finds more big caverns per dollar	"the third rental finds 1.9 times the first box's caverns"	counted caverns on the tail curve	3, 4

Commands are run from `caves/`. Tools under `stats/` carry their build line in their first comment; on a rented box `fleet/setup5.sh NAME` builds all of them into `/root/caves/`.

---

1. Exact: the game's own code

`measure.sh` runs the flood fill of `java/CaveMeasure.java` on terrain made by the Minecraft 26.3 server jar itself. It is slow (a minute or two per large cavern) and it is the only ground truth.

```
echo "6008414183811625950 19079170 13 -7883600" | ./measure.sh 6      # SEED X Y Z, 6 threads
```

It prints `seed x z volume core x0 x1 y0 y1 z0 z1 at x y z chunks N seconds S`. **Pass:** volume, core and the six box numbers equal the line the search wrote in `s3.txt` for that cavern, digit for digit. A difference of one block is a failure of the native measurer, not noise.

- Every record, and every cavern quoted to anyone, is re-measured this way first. More than thirty have been, all identical.
- Large-biomes worlds use `seedcaves/measure_lb.sh` (the same tool on the large-biomes preset).
- `java/NoiseProbe.java` is the same idea one level down: single noises of the game against `cavenoise.hpp`.

---

2. Equivalent: old against new on the same inputs

Two standards. A **result** (a volume, a density) must be bit-identical. A **filter statistic** (a look, a gate number) may differ by rounding, within the tolerance listed, as long as its cut decides the same seeds.

what changed	run	passes when
exact measurer (`native/gamegen.hpp`, `native/cavemeasure.cpp`)	`tests/check_measure.sh` (27 s); on a box also `GPU_DENSITY=1 tests/check_measure.sh`	"13 caverns identical": volume, core, box and /tp of 13 caverns of 40M to 54.6M, each confirmed by the game code (the chunk count is left out: it depends on how far ahead chunks are fetched)
density functions on the card (`native/gpudensity.cu`, generated code)	`native/densitycheck SEED X Z`	"of N values compared 0 differ in their bits"
CPU looks (`cpuprelook2.hpp`)	`stats/lookcheck2 3000`	float looks differ by less than 1e-6; 16-bit looks: "0 decided differently" at each cut
per-octave tables (`tableseed.hpp`)	`stats/tablesplitcheck`	"0 of 4000 comparisons differ"
coarse look on the card (`gpulook2.cu`)	`gpucheck2 look`; without a card, `stats/look2host` runs the kernel's steps on the host	rms difference from the CPU look about 1e-5; "GPU only" and "CPU only" a handful in 20,000
first look on the card (`gputables.cu`)	`tablecheck 512 < seeds`, and again with `GPUTABLES_OLD=1`	the two outputs identical to four decimals; largest difference from the CPU under 0.0005
full gate (`analyticgate.hpp`, `percgate.hpp`)	`stats/gatecheck2 300`	all three numbers within 1e-6, except a histogram bin (2.4e-4) in at most a few seeds per thousand
scan kernel, terrain score (`gpuscan.cu`)	`gpucheck terrain`	"0 of them missing from the kernel's": no seed the exact score passes is lost
scan kernel, chain order	the gate for a minute each with `CHAIN=0` and `CHAIN=1`; read "per billion scanned" in its status line	both rates equal within counting noise (about 15,020 ± 3) and the same share past the host's exact tests
scanner's terrain on the card	`GPU_TERRAIN=1 GPU_TERRAIN_CHECK=1 ./cavepairs5 SEED 200000 600 1 1250 292`	"on the GPU only 0, on the CPU only 0"
scanner's tables, formula stage on the card	the same seeds or sites with and without `GPU_TABLES=1` / `GPU_FORMULA=1`, one thread	identical site lists; identical F and O

Three things that have gone wrong and are now part of the procedure:

- **Compare scanner output with one thread.** With several, ties come out in a different order and the lists differ though nothing is wrong.
- **Time every kernel after a rebuild, not only check its numbers.** The scan kernel compiled with `-arch=sm_89` gives correct survivors at a fifth of the speed (1.8e10 instead of 8.2e10 seeds a second, gate alone on a 4080 Super). The gate's status line shows it; `setup5.sh` builds that one kernel without the flag.

- **Seeds are signed 64-bit numbers.** Since the chain scan half of them are negative. Anything new that reads a seed is tried on a negative one.

---

3. Better, quickly: rates on the box

`./rate.sh 240` (on the box) reports what the pipeline did over the next 240 seconds:

```
240 s, 46 workers: 888345 looks/s, 2859 listed/min of which top 420/min, 908 seeds/min,
44.1 CPU threads, held back 0.0% of the time; index 571.9
```

- **looks/s**: seeds past the scan. **top/min**: seeds the gate lists with a final score of -0.62 or better, which is what the looks are for and stays comparable when a gate setting changes what one look is worth. **seeds/min**: seeds the workers finish.
- **held back**: time the container was frozen by its CPU quota. Above a few percent the setting is wrong whatever else it shows (more runnable threads than the quota freeze all of them, the one driving the card too).
- **index** = top^0.6 × seeds^0.4, the fitted yield model. It ranks settings; it does not measure yield.

Procedure for one change:

1. `./rate.sh 120` for the state before, in the same hour.
2. Change one thing. Restart only the side it touches (`ONLY_GATE=1` or `ONLY_WORKERS=1` with `start_g14.sh`, which waits 75 seconds before it returns).
3. `./rate.sh 240`. Two readings of the same state differ by about 3%, so a difference under 5% is repeated or left for the curve.
4. Write the change, both readings and “from seed line N” into `runs/<run>/settings.txt`. Section 4 needs that line.

What this cannot show: one round of changes raised the index by 9% and the caverns per dollar not at all (581 ± 36 after, 661 ± 56 before). Faster workers had taken card time from the gate, and the model did not see it.

---

4. Better, for real: caverns on the tail curve

The measure the search is judged by is distinct caverns of at least V per dollar, and what follows from it, the largest cavern to expect from a budget.

```
python3 stats/fit_dollar.py stats/fit_dollar.png \
        stats/now/g13 2026-10-05T22:17:20Z stats/now/g14 2026-10-06T02:07:26Z      # every build,
per dollar
python3 stats/tail_curve.py       # the three boxes, and budget against expected largest cavern
```

- A cavern belongs to the build whose workers scanned its seed: the stretches come from the “from seed line N” entries of `settings.txt`, and cost is hours times the box’s price.
- Counts are Poisson: n caverns carry a relative error of 1/√n. At 230 caverns of 40M+ an hour, one hour gives ±7% at 40M+ and about ±25% at 46M+. A build is called better only if its level differs by more than two such errors; otherwise the report says “not shown to have moved”.

- The slope is fitted by maximum likelihood from 40M up (a factor 10 per ln 10 × mean(V − 40M)). Builds' own slopes have agreed within their errors so far, so levels are compared at one pooled slope, and a factor f in caverns per dollar is worth slope × log10 f on the largest cavern of any budget.
- Below 38M the counts are not comparable between builds (the exact rule leaves caverns out), so nothing is read there.
- A box under test is left alone while it is measured: a restart in the middle splits the count between two builds.

---

5. Calibrating a cut or a rule

A cut is judged on what it keeps of the big caverns and what work it saves.

1. **Use caverns found without the cut.** Either from runs before it existed or from a backfill that measured what an old rule skipped. A sample that already passed the cut, or that is ranked by a score the cut is part of, flatters it.
2. **Report kept k of n by size class** (40M+, 44M+, 46M+) next to the share of the work that remains.
3. **Fit on one half, judge on the other** when the rule has fitted weights: 200 random halves, and report the held-out gain, not the in-sample one.
4. **Record what did not work**, with the number, so it is not tried again.

Both mistakes this procedure exists for were made once. A tighter last look was estimated to lose 0.25% from the best 400 seeds by score; on 66 caverns of 40M+ found before the look existed it lost 3. An exact rule was reported as keeping 16 of 17 caverns of 42M+; on all caverns measured before it, it kept 48 of 64 of 40M+.

---

6. Operations

- **Before a box runs unattended:** the guard is running with its cap (`fleet/guard4.sh CAP 2.5` , pid in `fleet/guard.pid` , start mark in `state/spend_t0`); gate, `gpusrv` and the workers are up; `./rate.sh 60` shows no time held back. The guard runs on the laptop, so a sleeping laptop stops nothing until it wakes.
- **At the cap** the guard stops the search, packs the results, fetches them in parts checked by sha1, unpacks them into `final/<box>/` and only then destroys the instance. A failed download leaves the instance running and logs `RUN-KEPT` .
- **Before deleting an unpacked download:** `xz -t all.txz` , and the archive's file list compared with the files on disk.
- **Only instances labelled `cv-*` are ever touched.**

7. Before saying it

before	check
deploying a changed measurer or kernel	its row of section 2, on the box it will run on, and its speed
reporting a rate gain	before and after from `rate.sh` in the same hour, and time held back
reporting a yield gain	caverns counted per dollar for both builds, with their counts (section 4)
quoting a cavern or a record	`measure.sh` gives the same volume and box
trusting a new cut	kept k of n on caverns found without it (section 5)